Explainable Deep Learning Framework for Discovery and Optimization of Stronger Composite Materials

Pipeline Overview:

1. Load & align CFRP dataset + Material Strength CSV
2. SEM image subset loading + CNN encoder (unsupervised)
3. Multimodal fusion (tabular + SEM embeddings)
4. Multi-task prediction (Tensile Strength, Fracture Toughness, Structural Integrity Class)
5. Explainability (SHAP + Grad-CAM)
6. Material optimization module

> **Instructions:** Cells marked with `# ← ADD YOUR DATASET HERE` are placeholders. Run all other cells first, then fill in your paths.

0. Install Dependencies

```
!pip install shap scikit-learn torch torchvision pandas numpy matplotlib seaborn oper
```

1. Imports & Config

```
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
import torchvision.transforms as transforms
import torchvision.models as models
from PIL import Image
from tqdm import tqdm

from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, c
import shap

# ── Reproducibility ──────────────────────────────────────────────
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
print(f'Using device: {DEVICE}')

# ── Global Hyperparameters ──────────────────────────────────────────────
SEM_EMBED_DIM   = 128   # CNN encoder output size
BATCH_SIZE      = 32
EPOCHS_SEM      = 20    # SEM autoencoder pretraining
EPOCHS_MAIN     = 50    # Main multimodal model
LR              = 1e-3
SEM_SUBSET_SIZE = 800   # Use 500–1000 images from your 4.6 GB SEM dataset
IMG_SIZE        = 128   # Resize SEM images to 128×128
```

```
Using device: cuda
```

---

2. Dataset A — CFRP Dataset

> **← ADD YOUR DATASET HERE**
> Set `CFRP_CSV_PATH` to the path of your CFRP `.csv` file.
> Expected columns (rename if needed): see mapping table in cell below.

```
# ← ADD YOUR DATASET HERE
CFRP_CSV_PATH = '/content/CFRP_dataset.csv'   # <── CHANGE THIS

# ──────────────────────────────────────────────────────────────────────────
# Column name mapping: map YOUR column names → standardized internal names
# Edit the right-hand side to match your actual column headers.
CFRP_COL_MAP = {
    # Input features
    'Fiber_Volume_Fraction' : 'Fiber_Volume_Fraction',   # float, 0–1
    'Void_Content'          : 'Void_Content',            # float, %
    'Fiber_Orientation'     : 'Fiber_Orientation',       # float, degrees
    'Crack_Length'          : 'Crack_Length',            # float, mm
    'Defect_Density'        : 'Defect_Density',          # float
    'Temperature'           : 'Temperature',             # float, °C
    'Data_Type'             : 'Data_Type',               # categorical
    'Condition'             : 'Condition',               # categorical
    'Damage_Type'           : 'Damage_Type',             # categorical
    # Targets
    'Tensile_Strength'      : 'Tensile_Strength',        # float, MPa
    'Fracture_Toughness'    : 'Fracture_Toughness',      # float, MPa√m
    'Structural_Integrity'  : 'Structural_Integrity',    # int class label
}
# ──────────────────────────────────────────────────────────────────────────

cfrp_df = pd.read_csv(CFRP_CSV_PATH)
cfrp_df.rename(columns={v: k for k, v in CFRP_COL_MAP.items()}, inplace=True)
cfrp_df['source'] = 'CFRP'
print('CFRP shape:', cfrp_df.shape)
cfrp_df.head()
```

```
CFRP shape: (8080, 19)
```

	Sample_ID	Data_Type	Condition	Damage_Type	Vent_Holes	Fiber_Volume_Fraction	Void_Content
0	1	Thermograph	Mixed	NaN	4	0.566	0.058
1	2	C-scan (BVID with Holes)	Damaged	BVID	5	0.601	0.050
2	3	C-scan (Pristine with Holes)	Pristine	NaN	3	0.693	0.069
3	4	C-scan (BVID with Holes)	Damaged	BVID	6	0.501	0.047
4	5	C-scan (Pristine with Holes)	Pristine	NaN	2	0.502	0.010

Next steps: Generate code with `cfrp_df` New interactive sheet

---

3. Dataset B — Material Strength CSV (Domain Expansion)

> **← ADD YOUR DATASET HERE**
> Set `STRENGTH_CSV_PATH` to your strength dataset path.
> Edit `STRENGTH_COL_MAP` to align column names to the same schema.

```
# ← ADD YOUR DATASET HERE
STRENGTH_CSV_PATH = '/content/composite_material_strength.csv'   # <── CHANGE THIS

# ──────────────────────────────────────────────────────────────────────────
# Map YOUR column names → same standardized names as CFRP
# Only include columns that exist in your strength dataset.
# Fill missing ones with NaN after loading.
STRENGTH_COL_MAP = {
    'Fiber_Volume_Fraction' : 'Fiber_Volume_Fraction',
    'Void_Content'          : 'Void_Content',
    'Fiber_Orientation'     : 'Fiber_Orientation',
    'Crack_Length'          : 'Crack_Length',
    'Defect_Density'        : 'Defect_Density',
    'Temperature'           : 'Temperature',
    'Tensile_Strength'      : 'Tensile_Strength',
    'Fracture_Toughness'    : 'Fracture_Toughness',
    'Structural_Integrity'  : 'Structural_Integrity',
}
# ──────────────────────────────────────────────────────────────────────────

strength_df = pd.read_csv(STRENGTH_CSV_PATH)
strength_df.rename(columns={v: k for k, v in STRENGTH_COL_MAP.items()}, inplace=True)
strength_df['source'] = 'Strength'
print('Strength shape:', strength_df.shape)
strength_df.head()
```

```
Strength shape: (10000, 9)
```

	fiber_type	resin_type	density_g_cm3	layer_count	curing_temperature_c	fiber_volume_fractior
0	Aramid	Phenolic	2.16	16	138.0	0.56
1	Glass	Phenolic	1.51	8	62.1	0.69
2	Carbon	Polyester	1.55	6	90.4	0.53
3	Carbon	Phenolic	1.63	21	83.3	0.38
4	Basalt	Vinyl Ester	2.19	8	146.0	0.59

Next steps: Generate code with `strength_df` New interactive sheet

---

4. Dataset C — SEM Images

> **← ADD YOUR DATASET HERE**
> Set `SEM_IMAGE_DIR` to the folder containing your `.jpg`/`.png` SEM images.
> We randomly sample `SEM_SUBSET_SIZE` (default 800) from the full 4.6 GB dataset.

```
# ← ADD YOUR DATASET HERE
SEM_IMAGE_DIR = 'path/to/your/sem_images/'    # <── CHANGE THIS
# ──────────────────────────────────────────────────────────────────────

VALID_EXTS = {'.jpg', '.jpeg', '.png', '.tif', '.tiff', '.bmp'}
all_sem_paths = [
    os.path.join(SEM_IMAGE_DIR, f)
    for f in os.listdir(SEM_IMAGE_DIR)
    if os.path.splitext(f)[1].lower() in VALID_EXTS
]

print(f'Total SEM images found : {len(all_sem_paths)}')

# Random subset
random.shuffle(all_sem_paths)
sem_paths = all_sem_paths[:SEM_SUBSET_SIZE]
print(f'Using subset           : {len(sem_paths)} images')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!mv /content/5820067 /content/drive/MyDrive/
```

```
mv: cannot stat '/content/5820067': No such file or directory
```

```
extract_path = "/content/5820067"
```

```
!unzip -q /content/drive/MyDrive/5820067.zip -d /content/5820067/ '*.png'
```

```
replace /content/5820067/Fig1b_H1_HMG03-Fig3CompositeSpecimen.png? [y]es, [n]o, [A]ll, [N]one, [r]e
replace /content/5820067/Fig6_H2_HMG04-Fig3CompositeSpecimen.png? [y]es, [n]o, [A]ll, [N]one, [r]er
replace /content/5820067/Fig6_L1_HMG01-Fig5ContourVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig7_FigComp-Hist.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig1c_H1_HMG03-FigXOverlayVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
```

```
replace /content/5820067/Fig1b_L1_HMG01-Fig3CompositeSpecimen.png? [y]es, [n]o, [A]ll, [N]one, [r]
replace /content/5820067/Fig6_L2_HMG02-Fig5ContourVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig1b_L2_HMG02-Fig3CompositeSpecimen.png? [y]es, [n]o, [A]ll, [N]one, [r]
replace /content/5820067/Fig1c_L1_HMG01-FigXOverlayVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig1b_H2_HMG04-FigXOverlayVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig1c_L2_HMG02-FigXOverlayVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig1c_H2_HMG04-Fig5ContourVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/Fig6_H1_HMG03-Fig5ContourVf.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace /content/5820067/L1_HMG01-MaskBundles.png? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
```

---

5. Data Alignment & Combined Tabular Dataset

```
# All standardized feature columns (superset)
ALL_FEATURES = [
    'Fiber_Volume_Fraction', 'Void_Content', 'Fiber_Orientation',
    'Crack_Length', 'Defect_Density', 'Temperature',
    'Data_Type', 'Condition', 'Damage_Type'
]
TARGET_COLS = ['Tensile_Strength', 'Fracture_Toughness', 'Structural_Integrity']

# Add missing columns as NaN before concat
for col in ALL_FEATURES + TARGET_COLS:
    if col not in cfrp_df.columns:     cfrp_df[col]     = np.nan
    if col not in strength_df.columns: strength_df[col] = np.nan

combined_df = pd.concat([cfrp_df, strength_df], ignore_index=True)
print('Combined dataset shape:', combined_df.shape)
combined_df[ALL_FEATURES + TARGET_COLS].describe()
```

```
Combined dataset shape: (18080, 31)
```

	Fiber_Volume_Fraction	Void_Content	Fiber_Orientation	Crack_Length	Defect_Density	Tempe
count	8080.000000	8080.000000	0.0	0.0	8080.000000	
mean	0.574221	0.045028	NaN	NaN	0.087363	
std	0.072059	0.020318	NaN	NaN	0.053448	
min	0.450000	0.010000	NaN	NaN	0.010000	
25%	0.512000	0.028000	NaN	NaN	0.038000	
50%	0.575000	0.045000	NaN	NaN	0.086000	
75%	0.636000	0.063000	NaN	NaN	0.130000	
max	0.700000	0.080000	NaN	NaN	0.200000	

6. Preprocessing & Feature Engineering

```
# ── Categorical encoding ──────────────────────────────────────────
CAT_COLS = ['Data_Type', 'Condition', 'Damage_Type']
NUM_COLS = ['Fiber_Volume_Fraction', 'Void_Content', 'Fiber_Orientation',
            'Crack_Length', 'Defect_Density', 'Temperature']

le_dict = {}
for col in CAT_COLS:
    if combined_df[col].dtype == object:
        combined_df[col] = combined_df[col].fillna('Unknown')
        le = LabelEncoder()
```

```
        combined_df[col] = le.fit_transform(combined_df[col])
        le_dict[col] = le

# ── Impute numeric ──────────────────────────────────────────────────
for col in NUM_COLS:
    combined_df[col] = combined_df[col].fillna(combined_df[col].median())

# ── Interaction features ────────────────────────────────────────────
combined_df['Fiber_x_Void']    = combined_df['Fiber_Volume_Fraction'] * combined_df[
combined_df['Temp_x_Defect']   = combined_df['Temperature'] * combined_df['Defect_Der
combined_df['Strength_Defect_Ratio'] = (
    combined_df['Tensile_Strength'] / (combined_df['Defect_Density'] + 1e-8)
).fillna(0)

ENGINEERED_COLS = NUM_COLS + CAT_COLS + ['Fiber_x_Void', 'Temp_x_Defect']

# ── Target cleanup ──────────────────────────────────────────────────
combined_df.dropna(subset=['Tensile_Strength', 'Fracture_Toughness', 'Structural_Inte
combined_df['Structural_Integrity'] = combined_df['Structural_Integrity'].astype(int)
n_classes = combined_df['Structural_Integrity'].nunique()

print(f'Final dataset rows   : {len(combined_df)}')
print(f'Integrity classes    : {n_classes}')
print(f'Feature columns      : {len(ENGINEERED_COLS)}')
```

```
Final dataset rows   : 0
Integrity classes    : 0
Feature columns      : 11
```

```
# ── Scale ───────────────────────────────────────────────────────────
scaler = StandardScaler()
X_tab = scaler.fit_transform(combined_df[ENGINEERED_COLS].values.astype(np.float32))

y_reg = combined_df[['Tensile_Strength', 'Fracture_Toughness']].values.astype(np.float
y_cls = combined_df['Structural_Integrity'].values.astype(np.int64)

# Normalize regression targets
reg_scaler = StandardScaler()
y_reg = reg_scaler.fit_transform(y_reg)

print('X_tab shape:', X_tab.shape)
print('y_reg shape:', y_reg.shape)
print('y_cls shape:', y_cls.shape)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1888/3264853082.py in <cell line: 0>()
      1 # ── Scale ──────────────────────────────────────────────────────────────
      2 scaler = StandardScaler()
----> 3 X_tab = scaler.fit_transform(combined_df[ENGINEERED_COLS].values.astype(np.float32))
      4
      5 y_reg = combined_df[['Tensile_Strength', 'Fracture_Toughness']].values.astype(np.float32)

                                 6 frames
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py in check_array(array,
accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, force_all_finite,
ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples,
ensure_min_features, estimator, input_name)
   1128         n_samples = _num_samples(array)
   1129         if n_samples < ensure_min_samples:
-> 1130             raise ValueError(
   1131                 "Found array with %d sample(s) (shape=%s) while a"
   1132                 " minimum of %d is required%s."

ValueError: Found array with 0 sample(s) (shape=(0, 11)) while a minimum of 1 is required by
StandardScaler.
```

Next steps: Explain error

---

7. SEM Image Pipeline — CNN Autoencoder (Unsupervised)

```
# ── SEM Dataset class ──────────────────────────────────────────────────
class SEMDataset(Dataset):
    def __init__(self, paths, transform=None):
        self.paths     = paths
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        img = Image.open(self.paths[idx]).convert('L')   # grayscale SEM
        img = img.convert('RGB')                          # repeat to 3ch for backbor
        if self.transform:
            img = self.transform(img)
        return img


sem_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])

sem_dataset = SEMDataset(sem_paths, transform=sem_transform)
sem_loader  = DataLoader(sem_dataset, batch_size=BATCH_SIZE, shuffle=True, num_worker
print(f'SEM DataLoader ready: {len(sem_dataset)} images, {len(sem_loader)} batches')
```

```
# ── Visualize a few SEM images ─────────────────────────────────────────
fig, axes = plt.subplots(1, 5, figsize=(15, 3))
for i, ax in enumerate(axes):
```

```
    img = Image.open(sem_paths[i]).convert('L')
    ax.imshow(img, cmap='gray')
    ax.axis('off')
    ax.set_title(f'SEM #{i+1}')
plt.suptitle('Sample SEM Images', fontsize=14)
plt.tight_layout()
plt.show()
```

```
# ── CNN Autoencoder ──────────────────────────────────────────────────
class SEMEncoder(nn.Module):
    """Lightweight CNN encoder. Outputs SEM_EMBED_DIM-dim vector per image."""
    def __init__(self, embed_dim=SEM_EMBED_DIM):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32,  3, 2, 1), nn.ReLU(),   # 64
            nn.Conv2d(32, 64, 3, 2, 1), nn.ReLU(),   # 32
            nn.Conv2d(64, 128,3, 2, 1), nn.ReLU(),   # 16
            nn.Conv2d(128,256,3, 2, 1), nn.ReLU(),   # 8
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(256, embed_dim)
        )

    def forward(self, x):
        return self.encoder(x)


class SEMAutoencoder(nn.Module):
    def __init__(self, embed_dim=SEM_EMBED_DIM):
        super().__init__()
        self.enc = SEMEncoder(embed_dim)
        self.decoder = nn.Sequential(
            nn.Linear(embed_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 3 * IMG_SIZE * IMG_SIZE),
            nn.Tanh()
        )

    def forward(self, x):
        z   = self.enc(x)
        out = self.decoder(z).view(-1, 3, IMG_SIZE, IMG_SIZE)
        return out, z


sem_ae    = SEMAutoencoder().to(DEVICE)
ae_opt    = optim.Adam(sem_ae.parameters(), lr=LR)
ae_loss_fn = nn.MSELoss()
print(sem_ae)
```

```
# ── Phase 1: Train SEM Autoencoder ──────────────────────────────────
ae_losses = []

for epoch in range(EPOCHS_SEM):
    sem_ae.train()
    total_loss = 0
    for imgs in tqdm(sem_loader, desc=f'AE Epoch {epoch+1}/{EPOCHS_SEM}', leave=Fals
        imgs = imgs.to(DEVICE)
        recon, _ = sem_ae(imgs)
        loss = ae_loss_fn(recon, imgs)
```

```
            ae_opt.zero_grad()
            loss.backward()
            ae_opt.step()
            total_loss += loss.item()
        avg = total_loss / len(sem_loader)
        ae_losses.append(avg)
        if (epoch+1) % 5 == 0:
            print(f'  Epoch {epoch+1:3d} | AE Loss: {avg:.5f}')

    plt.figure(figsize=(8,3))
    plt.plot(ae_losses, color='steelblue')
    plt.title('SEM Autoencoder Training Loss')
    plt.xlabel('Epoch'); plt.ylabel('MSE Loss')
    plt.tight_layout(); plt.show()
```

```
    # ── Extract SEM Embeddings for ALL subset images ──────────────────────
    sem_ae.eval()
    all_embeddings = []

    extract_transform = transforms.Compose([
        transforms.Resize((IMG_SIZE, IMG_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize([0.5]*3, [0.5]*3)
    ])
    extract_dataset = SEMDataset(sem_paths, transform=extract_transform)
    extract_loader  = DataLoader(extract_dataset, batch_size=BATCH_SIZE, shuffle=False)

    with torch.no_grad():
        for imgs in extract_loader:
            _, z = sem_ae(imgs.to(DEVICE))
            all_embeddings.append(z.cpu().numpy())

    all_embeddings = np.vstack(all_embeddings)   # shape: (N_sem, SEM_EMBED_DIM)
    print('SEM embeddings shape:', all_embeddings.shape)

    # ── Aggregate SEM embeddings into a SINGLE global descriptor ──────────
    # (used when tabular rows >> SEM images; broadcast mean embedding)
    mean_sem_embedding = all_embeddings.mean(axis=0, keepdims=True)  # (1, 128)
    print('Mean SEM embedding shape:', mean_sem_embedding.shape)
```

---

8. Multimodal Fusion & Multi-Task Model

```
    # ── Build fused feature matrix ────────────────────────────────────────
    # Strategy: Broadcast the mean SEM embedding to every tabular row.
    # If you have per-sample SEM images (matched IDs), replace this with a
    # per-row lookup instead of broadcasting.

    N_rows = X_tab.shape[0]
    X_sem_broadcast = np.tile(mean_sem_embedding, (N_rows, 1))  # (N, 128)
    X_fused = np.hstack([X_tab, X_sem_broadcast])               # (N, tab_dim + 128)

    print('Fused feature matrix shape:', X_fused.shape)

    # ── Train/val/test split ──────────────────────────────────────────────
    X_tr, X_te, yr_tr, yr_te, yc_tr, yc_te = train_test_split(
        X_fused, y_reg, y_cls, test_size=0.15, random_state=SEED
    )
```

```
X_tr, X_val, yr_tr, yr_val, yc_tr, yc_val = train_test_split(
    X_tr, yr_tr, yc_tr, test_size=0.15, random_state=SEED
)

print(f'Train: {X_tr.shape[0]}  Val: {X_val.shape[0]}  Test: {X_te.shape[0]}')

def to_tensor(*arrays):
    return [torch.tensor(a) for a in arrays]

Xtr_t, yr_tr_t, yc_tr_t = to_tensor(X_tr.astype(np.float32),
                                    yr_tr.astype(np.float32),
                                    yc_tr.astype(np.int64))
Xv_t,  yr_v_t,  yc_v_t  = to_tensor(X_val.astype(np.float32),
                                    yr_val.astype(np.float32),
                                    yc_val.astype(np.int64))
Xte_t, yr_te_t, yc_te_t = to_tensor(X_te.astype(np.float32),
                                    yr_te.astype(np.float32),
                                    yc_te.astype(np.int64))

train_dl = DataLoader(TensorDataset(Xtr_t, yr_tr_t, yc_tr_t), batch_size=BATCH_SIZE,
val_dl   = DataLoader(TensorDataset(Xv_t,  yr_v_t,  yc_v_t),  batch_size=BATCH_SIZE)
test_dl  = DataLoader(TensorDataset(Xte_t, yr_te_t, yc_te_t), batch_size=BATCH_SIZE)
```

```
# ── Multi-task DNN ──────────────────────────────────────────────────
class MultiTaskNet(nn.Module):
    def __init__(self, in_dim, n_classes):
        super().__init__()
        # Shared backbone
        self.shared = nn.Sequential(
            nn.Linear(in_dim, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512, 256),    nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128),    nn.ReLU()
        )
        # Regression heads
        self.head_tensile    = nn.Sequential(nn.Linear(128, 64), nn.ReLU(), nn.Linear
        self.head_fracture   = nn.Sequential(nn.Linear(128, 64), nn.ReLU(), nn.Linear
        # Classification head
        self.head_integrity  = nn.Sequential(nn.Linear(128, 64), nn.ReLU(), nn.Linear

    def forward(self, x):
        z  = self.shared(x)
        ts = self.head_tensile(z).squeeze(-1)
        ft = self.head_fracture(z).squeeze(-1)
        si = self.head_integrity(z)
        return ts, ft, si


model = MultiTaskNet(in_dim=X_fused.shape[1], n_classes=n_classes).to(DEVICE)
print(model)
print(f'Parameters: {sum(p.numel() for p in model.parameters()):,}')
```

```
# ── Training loop ───────────────────────────────────────────────────
optimizer  = optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler  = optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5, factor=0.5)
mse_fn     = nn.MSELoss()
ce_fn      = nn.CrossEntropyLoss()

# Loss weights (tune if needed)
W_REG, W_CLS = 1.0, 0.5
```

```
train_losses, val_losses = [], []
best_val_loss = float('inf')

for epoch in range(EPOCHS_MAIN):
    model.train()
    t_loss = 0
    for xb, yr_b, yc_b in train_dl:
        xb, yr_b, yc_b = xb.to(DEVICE), yr_b.to(DEVICE), yc_b.to(DEVICE)
        ts, ft, si = model(xb)
        loss = (W_REG * mse_fn(ts, yr_b[:,0])
              + W_REG * mse_fn(ft, yr_b[:,1])
              + W_CLS * ce_fn(si, yc_b))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        t_loss += loss.item()

    model.eval()
    v_loss = 0
    with torch.no_grad():
        for xb, yr_b, yc_b in val_dl:
            xb, yr_b, yc_b = xb.to(DEVICE), yr_b.to(DEVICE), yc_b.to(DEVICE)
            ts, ft, si = model(xb)
            v_loss += (W_REG * mse_fn(ts, yr_b[:,0])
                     + W_REG * mse_fn(ft, yr_b[:,1])
                     + W_CLS * ce_fn(si, yc_b)).item()

    train_losses.append(t_loss / len(train_dl))
    val_losses.append(v_loss / len(val_dl))
    scheduler.step(val_losses[-1])

    if val_losses[-1] < best_val_loss:
        best_val_loss = val_losses[-1]
        torch.save(model.state_dict(), 'best_model.pth')

    if (epoch+1) % 10 == 0:
        print(f'Epoch {epoch+1:3d} | Train: {train_losses[-1]:.4f} | Val: {val_losses

# Plot
plt.figure(figsize=(9,3))
plt.plot(train_losses, label='Train')
plt.plot(val_losses,   label='Val')
plt.legend(); plt.title('Multi-Task Training Loss')
plt.xlabel('Epoch'); plt.ylabel('Combined Loss')
plt.tight_layout(); plt.show()
```

---

9. Evaluation

```
# Load best checkpoint
model.load_state_dict(torch.load('best_model.pth', map_location=DEVICE))
model.eval()

all_ts, all_ft, all_si_pred, all_si_true = [], [], [], []
all_yr = []

with torch.no_grad():
    for xb, yr_b, yc_b in test_dl:
        ts, ft, si = model(xb.to(DEVICE))
        all_ts.append(ts.cpu().numpy())
```

```
            all_ft.append(ft.cpu().numpy())
            all_si_pred.append(si.argmax(1).cpu().numpy())
            all_si_true.append(yc_b.numpy())
            all_yr.append(yr_b.numpy())

    pred_ts   = np.concatenate(all_ts)
    pred_ft   = np.concatenate(all_ft)
    pred_cls  = np.concatenate(all_si_pred)
    true_cls  = np.concatenate(all_si_true)
    true_reg  = np.vstack(all_yr)

    rmse_ts  = mean_squared_error(true_reg[:,0], pred_ts, squared=False)
    r2_ts    = r2_score(true_reg[:,0], pred_ts)
    rmse_ft  = mean_squared_error(true_reg[:,1], pred_ft, squared=False)
    r2_ft    = r2_score(true_reg[:,1], pred_ft)
    acc      = accuracy_score(true_cls, pred_cls)
    f1       = f1_score(true_cls, pred_cls, average='weighted')

    print('='*50)
    print('         TEST SET RESULTS')
    print('='*50)
    print(f'Tensile Strength   RMSE: {rmse_ts:.4f}  R²: {r2_ts:.4f}')
    print(f'Fracture Toughness RMSE: {rmse_ft:.4f}  R²: {r2_ft:.4f}')
    print(f'Structural Integrity Acc: {acc:.4f}  F1: {f1:.4f}')
    print()
    print(classification_report(true_cls, pred_cls, zero_division=0))
```

```
    # ── Prediction vs Truth plots ──────────────────────────────────────
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    for ax, pred, true, title in zip(
        axes,
        [pred_ts, pred_ft],
        [true_reg[:,0], true_reg[:,1]],
        ['Tensile Strength', 'Fracture Toughness']
    ):
        ax.scatter(true, pred, alpha=0.5, s=10)
        lims = [min(true.min(), pred.min()), max(true.max(), pred.max())]
        ax.plot(lims, lims, 'r--')
        ax.set_title(f'{title}\nR²={r2_score(true, pred):.3f}')
        ax.set_xlabel('True'); ax.set_ylabel('Predicted')

    plt.tight_layout(); plt.show()
```

---

10. Explainability — SHAP (Tabular)

```
    # We explain the TABULAR portion only (first len(ENGINEERED_COLS) features)
    # using a wrapper that returns tensile strength prediction

    model.eval()

    def predict_tensile(X_np):
        """Wrapper: takes numpy (N, fused_dim), returns tensile strength (N,)"""
        with torch.no_grad():
            X_t = torch.tensor(X_np.astype(np.float32)).to(DEVICE)
            ts, _, _ = model(X_t)
        return ts.cpu().numpy()
```

```
# Use a small background set for SHAP
bg_size = min(100, X_tr.shape[0])
background = X_tr[:bg_size]
explain_set = X_te[:200]

explainer    = shap.KernelExplainer(predict_tensile, background)
shap_values  = explainer.shap_values(explain_set, nsamples=100)

# Feature names (tab + sem dims)
sem_names  = [f'SEM_dim_{i}' for i in range(SEM_EMBED_DIM)]
feat_names = ENGINEERED_COLS + sem_names

# Summary plot (tabular only for clarity)
shap.summary_plot(shap_values[:, :len(ENGINEERED_COLS)],
                  explain_set[:, :len(ENGINEERED_COLS)],
                  feature_names=ENGINEERED_COLS,
                  plot_type='bar',
                  show=True)
```

```
# Beeswarm plot
shap.summary_plot(shap_values[:, :len(ENGINEERED_COLS)],
                  explain_set[:, :len(ENGINEERED_COLS)],
                  feature_names=ENGINEERED_COLS)
```

---

11. Explainability — Grad-CAM (SEM Images)

```
# ── Grad-CAM on the SEM Autoencoder encoder ──────────────────────────
# We compute gradients of the embedding norm w.r.t. the last conv layer.

class GradCAM:
    def __init__(self, model, target_layer):
        self.model  = model
        self.grads  = None
        self.acts   = None
        target_layer.register_forward_hook(self._save_acts)
        target_layer.register_backward_hook(self._save_grads)

    def _save_acts(self, _, __, output): self.acts = output
    def _save_grads(self, _, __, grad_out): self.grads = grad_out[0]

    def compute(self, img_tensor):
        self.model.eval()
        out = self.model.enc.encoder[:8](img_tensor)   # up to last conv
        # Treat embedding norm as scalar loss
        _, emb = self.model(img_tensor)
        score  = emb.norm()
        self.model.zero_grad()
        score.backward()
        weights  = self.grads.mean(dim=(2,3), keepdim=True)
        cam      = (weights * self.acts).sum(dim=1, keepdim=True)
        cam      = torch.relu(cam).squeeze().detach().cpu().numpy()
        cam      = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        return cam


# Target layer: last Conv2d in encoder
```

```
target_layer = sem_ae.enc.encoder[6]   # Conv2d(128, 256, ...)
gradcam      = GradCAM(sem_ae, target_layer)

# Visualize on 4 random SEM images
fig, axes = plt.subplots(4, 2, figsize=(8, 14))
for i in range(4):
    img_path = sem_paths[i]
    raw = Image.open(img_path).convert('L').convert('RGB')
    img_t = extract_transform(raw).unsqueeze(0).to(DEVICE).requires_grad_(True)
    try:
        cam = gradcam.compute(img_t)
        import cv2
        raw_np = np.array(raw.resize((IMG_SIZE, IMG_SIZE)))
        heatmap = cv2.applyColorMap(
            cv2.resize((cam * 255).astype(np.uint8), (IMG_SIZE, IMG_SIZE)),
            cv2.COLORMAP_JET
        )
        overlay = cv2.addWeighted(raw_np, 0.6, heatmap, 0.4, 0)
        axes[i,0].imshow(raw_np, cmap='gray'); axes[i,0].set_title('Original')
        axes[i,1].imshow(overlay);             axes[i,1].set_title('Grad-CAM')
    except Exception as e:
        axes[i,0].set_title(f'Error: {e}')
    for ax in axes[i]: ax.axis('off')

plt.suptitle('Grad-CAM: SEM Microstructure Attention', fontsize=13)
plt.tight_layout(); plt.show()
```

---

12. Material Optimization Module

```
# ── Grid Search Optimization ──────────────────────────────────────────
# Sweep over Fiber Volume Fraction & Void Content; fix everything else at median.

model.eval()

fiber_range = np.linspace(0.3, 0.7, 20)
void_range  = np.linspace(0.0, 0.10, 20)

results = []
median_row = X_tab.mean(axis=0).copy()    # baseline

FV_IDX  = ENGINEERED_COLS.index('Fiber_Volume_Fraction')
VC_IDX  = ENGINEERED_COLS.index('Void_Content')

for fv in fiber_range:
    for vc in void_range:
        row = median_row.copy()
        row[FV_IDX] = fv
        row[VC_IDX] = vc
        # Fuse with mean SEM
        fused_row = np.hstack([row, mean_sem_embedding[0]]).astype(np.float32)
        xt = torch.tensor(fused_row).unsqueeze(0).to(DEVICE)
        with torch.no_grad():
            ts, ft, si = model(xt)
        # De-scale tensile strength
        ts_real = reg_scaler.inverse_transform([[ts.item(), 0]])[0][0]
        results.append({'Fiber_Volume': fv, 'Void_Content': vc,
                        'Pred_Tensile': ts_real, 'Pred_Class': si.argmax(1).item()})
```

```
opt_df = pd.DataFrame(results)
best   = opt_df.loc[opt_df['Pred_Tensile'].idxmax()]
print('Optimal configuration:')
print(best)
```

```
# ── Heatmap ──────────────────────────────────────────────────────────────
pivot = opt_df.pivot(index='Void_Content', columns='Fiber_Volume', values='Pred_Tensi

plt.figure(figsize=(9, 5))
sns.heatmap(pivot, cmap='YlOrRd', fmt='.0f', annot=False,
            xticklabels=np.round(fiber_range, 2),
            yticklabels=np.round(void_range, 3))
plt.title('Predicted Tensile Strength: Fiber Volume vs Void Content')
plt.xlabel('Fiber Volume Fraction'); plt.ylabel('Void Content')
plt.tight_layout(); plt.show()
```

```
# ── Bayesian Optimization ────────────────────────────────────────────────
from bayes_opt import BayesianOptimization

def objective(fiber_vol, void_content, temp, defect_density):
    row = median_row.copy()
    row[FV_IDX] = fiber_vol
    row[VC_IDX] = void_content
    row[ENGINEERED_COLS.index('Temperature')]    = temp
    row[ENGINEERED_COLS.index('Defect_Density')] = defect_density
    fused = np.hstack([row, mean_sem_embedding[0]]).astype(np.float32)
    xt = torch.tensor(fused).unsqueeze(0).to(DEVICE)
    with torch.no_grad():
        ts, _, _ = model(xt)
    return ts.item()   # maximize predicted tensile

pbounds = {
    'fiber_vol'       : (0.3, 0.7),
    'void_content'    : (0.0, 0.10),
    'temp'            : (20, 200),
    'defect_density'  : (0.0, 0.5),
}

optimizer_bo = BayesianOptimization(f=objective, pbounds=pbounds, random_state=SEED,
optimizer_bo.maximize(init_points=10, n_iter=30)

print('\nBayesian Optimization — Best Parameters:')
print(optimizer_bo.max)
```

---

13. Cross-Modal Explanation Alignment

```
# Link SEM Grad-CAM attention regions with SHAP importance of Void_Content
# This gives the "visual + numerical explanation alignment" from the paper.

void_shap_mean = np.abs(shap_values[:, VC_IDX]).mean()

print('Cross-Modal Explanation Alignment')
print('='*45)
print(f'SHAP importance of Void_Content      : {void_shap_mean:.4f}')
print(f'Grad-CAM highlights void-rich regions: See plots in Section 11')
print()
print('Interpretation: High SHAP importance of Void_Content + Grad-CAM')
```

```
print('focus on dark (void) regions confirms that void microstructure')
print('is a key driver of predicted mechanical performance.')

# Top-5 SHAP features
mean_abs_shap = np.abs(shap_values[:, :len(ENGINEERED_COLS)]).mean(axis=0)
top5 = pd.Series(mean_abs_shap, index=ENGINEERED_COLS).nlargest(5)
print('\nTop-5 SHAP Features:')
print(top5)
```

---

14. Save Artifacts

```
# Save model, scalers, SEM embeddings, optimization results
import pickle

torch.save(model.state_dict(), 'best_multitask_model.pth')
torch.save(sem_ae.state_dict(), 'sem_autoencoder.pth')

with open('scaler_tab.pkl', 'wb') as f:  pickle.dump(scaler, f)
with open('scaler_reg.pkl', 'wb') as f:  pickle.dump(reg_scaler, f)
with open('le_dict.pkl',    'wb') as f:  pickle.dump(le_dict, f)

np.save('sem_embeddings_subset.npy', all_embeddings)
opt_df.to_csv('optimization_grid_results.csv', index=False)

print('All artifacts saved.')
```

---

15. Summary Dashboard

```
print('\n' + '='*60)
print('  EXPLAINABLE DL FRAMEWORK — FINAL SUMMARY')
```